Boardroom AI

A Telegram-based multi-agent business advisory system built with **n8n**, **Google Gemini**, **Google Sheets**, and the **Telegram Bot API**.

Boardroom AI receives a business question through Telegram, assigns the request to relevant specialist agents, gathers independent analyses, and combines their findings into a single executive recommendation.

The project demonstrates practical multi-agent orchestration, structured AI outputs, reusable sub-workflows, conditional routing, conversation logging, rate-limit handling, and Telegram-based user interaction.

Table of Contents

Overview

Boardroom AI is a multi-agent advisory bot that allows users to submit business ideas, strategic questions, pricing challenges, launch plans, or operational problems through Telegram.

Instead of sending every request to one general-purpose AI prompt, the system uses a supervisor-and-specialist architecture.

The Supervisor Agent first analyses the request and decides which advisers should contribute. Selected specialist agents then review the problem independently from different perspectives.

The available specialists are:

- Market Research Analyst
- Business Strategy Consultant
- Financial Analyst
- Risk and Challenge Analyst

Their findings are then passed to an Executive Editor, which resolves overlaps, identifies contradictions, and produces one concise boardroom recommendation.

The final response is returned directly to the user in Telegram and stored in Google Sheets for auditability and portfolio demonstration.

Project Objective

The objective of Boardroom AI is to demonstrate how multiple AI agents can collaborate inside an automation workflow.

The project is designed to show:

1. Multi-agent orchestration
2. Dynamic agent selection
3. Reusable n8n sub-workflows
4. Structured Gemini outputs
5. Parallel or sequential specialist execution

6. Multi-item aggregation
7. Executive synthesis
8. Telegram interaction
9. Persistent logging
10. API quota and rate-limit management
11. Practical business use cases
12. Production-oriented workflow design

Key Features

Telegram interface

Users interact with the system through a Telegram bot.

The bot supports:

- normal business questions;
- `/quick` analysis;
- `/deep` analysis;
- `/help`;
- `/start`;
- greeting handling;
- appreciation replies;
- unsupported-message guidance.

Dynamic specialist selection

The Supervisor Agent selects the most relevant advisers based on the request.

For example:

```
User request:
Help me price an AI automation consulting service.

Selected agents:
strategy
finance
```

A broader request may use all four agents:

```
User request:
Analyse a laundry pickup and delivery service in Lagos.
```

```
Selected agents:
market
strategy
finance
risk
```

Reusable specialist workflow

The project uses one reusable specialist sub-workflow instead of four duplicate workflows.

The parent workflow sends:

```
agent_type
agent_name
agent_objective
user_request
request_type
analysis_depth
context
conversation_id
```

The specialist workflow changes its system instructions dynamically according to `agent_type`.

Structured AI responses

The Supervisor and Specialist agents use JSON Schema output parsers.

This ensures that later workflow nodes receive predictable fields such as:

```
selected_agents
agent_tasks
executive_summary
key_findings
recommendations
confidence
```

Executive synthesis

The Executive Editor combines all specialist analyses into one response.

It does not simply concatenate agent outputs. It is instructed to:

- merge related findings;
- resolve contradictions;
- separate assumptions from facts;
- highlight financial considerations;
- identify risks;
- produce an execution roadmap;
- give a final verdict.

Progress message editing

When a request is received, the bot sends:

```
🏛️ Boardroom assembled.

The supervisor is reviewing your request and assigning the most relevant
advisers…
```

After the analysis completes, the workflow edits that same Telegram message with the final recommendation.

Persistent logging

The project stores:

- conversations;
- specialist responses;
- execution metadata;
- activity history;
- confidence scores;
- selected agents;
- final responses.

Rate-limit protection

The workflow can execute specialists sequentially using:

```
Loop Over Items
Wait
Execute Sub-workflow
```

This prevents several Gemini requests from being sent at the same instant.

How It Works

A typical request follows this sequence:

```
Telegram User
      ↓
Telegram Trigger
      ↓
Message Normalisation
      ↓
Command and Message Classification
      ↓
Progress Message
      ↓
Supervisor Agent
      ↓
Supervisor Plan Validation
      ↓
Selected Agents Expanded
      ↓
Specialist Sub-Workflow Executions
      ↓
Specialist Responses Aggregated
      ↓
Executive Editor
      ↓
Telegram Response
      ↓
Google Sheets Logging
```

Example:

```
/deep I want to start a laundry pickup and delivery service in Lagos.
```

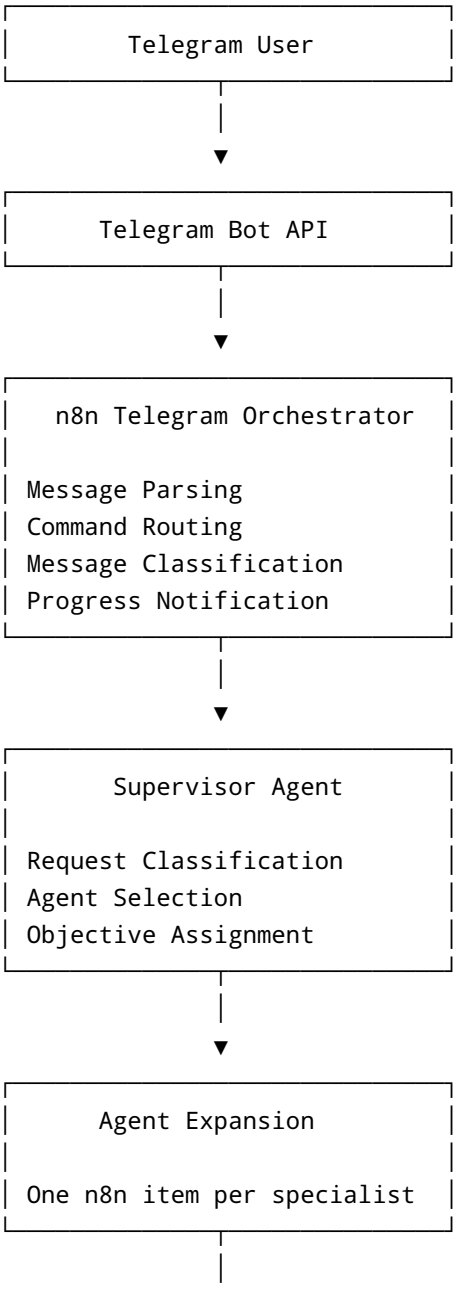
The Supervisor may return:

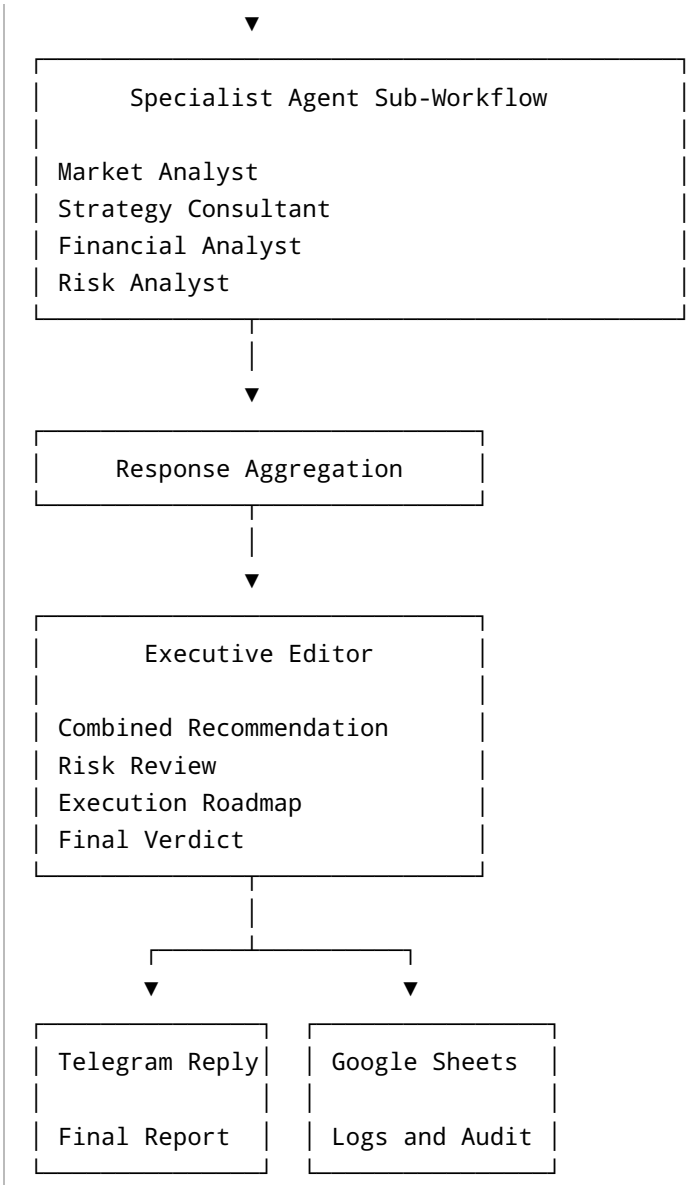
```
{
  "request_type": "business_launch",
  "analysis_depth": "deep",
  "selected_agents": [
    "market",
    "strategy",
    "finance",
```

```
    "risk"
  ]
}
```

The system then creates four items and sends each one to the Specialist Agent sub-workflow.

System Architecture





Agent Architecture

Supervisor Agent

The Supervisor Agent is responsible for orchestration.

It does not provide the final business answer.

Its responsibilities include:

- understanding the request;
- identifying the request type;
- summarising the problem;
- selecting the relevant specialists;
- creating one objective for every selected specialist;
- setting the analysis depth;
- providing shared context;
- describing the expected output.

Market Research Analyst

The Market Research Analyst focuses on:

- customer segments;
- demand;
- buying behaviour;
- market gaps;
- competitors;
- alternatives;
- positioning;
- differentiation;
- market validation requirements.

Business Strategy Consultant

The Strategy Agent focuses on:

- value proposition;
- business model;
- strategic priorities;
- go-to-market approach;
- implementation milestones;
- dependencies;
- competitive advantage;
- execution planning.

Financial Analyst

The Financial Analyst focuses on:

- pricing;
- startup costs;
- operating costs;
- revenue assumptions;
- contribution margins;

- unit economics;
- break-even drivers;
- cash-flow concerns;
- financial feasibility.

Financial figures are treated as illustrative assumptions unless verified data is supplied.

Risk and Challenge Analyst

The Risk Agent focuses on:

- optimistic assumptions;
- failure scenarios;
- operational risks;
- market risks;
- financial risks;
- legal or compliance considerations;
- implementation risks;
- likelihood and impact;
- practical mitigations;
- unanswered questions.

Executive Editor

The Executive Editor combines the specialist responses into one coherent answer.

Its response includes:

```
🏛️ BOARDROOM VERDICT

🎯 Opportunity

🔍 Key Findings

💰 Commercial View

⚠️ Major Risks

🗺️ Execution Roadmap

✅ Final Recommendation
```

Technology Stack

Automation

- n8n
- n8n external Python runners
- Docker
- Docker Compose

AI

- Google Gemini API
- n8n Basic LLM Chain
- n8n Structured Output Parser

Messaging

- Telegram Bot API
- n8n Telegram Trigger
- n8n Telegram node

Storage

- Google Sheets

Development

- Python
- JavaScript expressions
- JSON Schema
- Git
- GitHub
- PowerShell
- Cloudflare Tunnel

Workflow Structure

The project contains two primary n8n workflows.

Workflow 1

```
Boardroom AI — Telegram Orchestrator
```

This workflow handles Telegram interaction and multi-agent coordination.

Workflow 2

```
Boardroom AI — Specialist Agent
```

This reusable sub-workflow performs specialist analysis.

Telegram Orchestrator Workflow

Recommended node structure:

```
01 Telegram Trigger
        ↓
02 Normalise Telegram Update
        ↓
03 Is Supported Message?
        ↓
04 Prepare Conversation
        ↓
04B Route Command
        ↓
04C Classify Message
        ↓
04D Route Message Type
        ↓ advisory_request
05 Send Progress Message
        ↓
05B Restore Conversation Context
        ↓
06 Supervisor Planner
        ↓
07 Validate Supervisor Plan
        ↓
07B Restore Conversation Context
        ↓
08 Expand Selected Agents
```

```
        ↓
09 Execute Specialist Sub-workflow
        │
        ├── 10A Prepare Agent Log
        │       ↓
        │   11A Save Agent Response
        │
        └── 10 Aggregate Agent Responses
                ↓
            10B Restore Final Conversation Context
                ↓
            11 Executive Editor
                ↓
            12 Normalise Executive Response
                ↓
            12B Prepare Telegram Result
                ↓
            13 Edit Progress Message
                ↓
            14 Prepare Conversation Log
                ↓
            15 Save Conversation
                ↓
            16 Prepare Activity Record
                ↓
            17 Record Activity
                ↓
            18 Return Workflow Result
```

Specialist Agent Workflow

Recommended node structure:

```
01 Execute Sub-workflow Trigger
        ↓
02 Validate Specialist Request
        ↓
03 Prepare Specialist Prompt
        ↓
04 Specialist LLM Analysis
   ├── Google Gemini Chat Model
   └── Structured Output Parser
        ↓
```

```
05 Normalise Specialist Response
        ↓
05B Restore Specialist Metadata
        ↓
06 Return Specialist Result
```

Telegram Commands

`/start`

Introduces the bot and explains its purpose.

Example reply:

```
👋 Welcome to Boardroom AI.

Send a business idea, challenge, pricing decision, or launch plan and I will
assemble a team of specialist advisers.
```

`/help`

Displays the supported commands and example requests.

`/quick`

Uses the two most relevant agents.

Example:

```
/quick Help me create pricing packages for an AI automation consulting service.
```

`/deep`

Uses all four specialist agents.

Example:

```
/deep Analyse a laundry pickup and delivery business in Lagos.
```

Standard mode

A normal message without a command uses between two and four agents.

Example:

```
I want to start a subscription-based analytics service for small businesses.
```

Message Classification

Simple conversational messages should not reach the Supervisor Planner.

The workflow classifies incoming messages into:

```
greeting
gratitude
farewell
help_request
advisory_request
unsupported
```

Examples:

Message	Classification
Hello	greeting
Thanks	gratitude
Goodbye	farewell
What can you do?	help_request
Help me price my consulting service	advisory_request
Blank message	unsupported

Only `advisory_request` continues to the multi-agent workflow.

This prevents strict-output parsing errors caused by sending greetings to the Supervisor Agent.

Supervisor Agent

The Supervisor Agent receives:

```
request_mode
user_request
```

It returns:

```
request_type
analysis_depth
problem_summary
selected_agents
agent_tasks
key_context
expected_output
```

Example:

```
{
  "request_type": "pricing_strategy",
  "analysis_depth": "quick",
  "problem_summary": "The user needs a pricing model for an AI automation
consultancy.",
  "selected_agents": [
    "strategy",
    "finance"
  ],
  "agent_tasks": [
    {
      "agent_type": "strategy",
      "objective": "Design service packages and market positioning."
    },
    {
      "agent_type": "finance",
      "objective": "Assess pricing, delivery costs and profitability."
    }
  ],
  "key_context": "The target market is small Nigerian businesses.",
  "expected_output": "A practical pricing structure and implementation plan."
}
```

Specialist Agents

The specialist sub-workflow receives one item per selected agent.

Example input:

```
{
  "conversation_id": "BRD-20260731110530-123456789",
  "agent_type": "finance",
  "agent_name": "Financial Analyst",
  "agent_objective": "Assess pricing, delivery costs and profitability.",
  "user_request": "Help me price an AI automation service.",
  "request_type": "pricing_strategy",
  "analysis_depth": "quick",
  "context": "The target customers are small Nigerian businesses."
}
```

The sub-workflow changes the specialist instruction according to `agent_type`.

Specialist Output

Each specialist returns:

```
{
  "executive_summary": "A concise assessment from the specialist perspective.",
  "key_findings": [
    "Finding one",
    "Finding two"
  ],
  "recommendations": [
    "Recommendation one",
    "Recommendation two"
  ],
  "key_assumptions": [
    "Assumption one"
  ],
  "critical_questions": [
    "Question one"
  ],
  "strongest_insight": "The most valuable conclusion.",
```

```
  "confidence": 84
}
```

Metadata is then restored:

```
conversation_id
agent_type
agent_name
agent_objective
user_request
request_type
analysis_depth
created_at
```

Executive Editor

The Executive Editor receives the complete `agent_responses` array.

Example:

```
{
  "agent_count": 2,
  "selected_agents": [
    "strategy",
    "finance"
  ],
  "average_confidence": 84.5,
  "agent_responses": [
    {
      "agent_type": "strategy",
      "executive_summary": "..."
    },
    {
      "agent_type": "finance",
      "executive_summary": "..."
    }
  ]
}
```

The editor must:

- avoid repeating each specialist verbatim;

• merge related recommendations;
• identify disagreements;
• label financial assumptions;
• surface major risks;
• provide three to five implementation steps;
• remain below Telegram's practical message limit;
• end with a clear verdict.

Google Sheets Database

Create a workbook named:

```
Boardroom AI Database
```

Add these sheets:

```
Conversations
Agent_Responses
Activity_Log
```

Conversations Sheet

Headers:

```
conversation_id
chat_id
user_id
username
request_text
request_mode
request_type
selected_agents
analysis_depth
final_response
status
telegram_message_id
created_at
completed_at
execution_id
```

Purpose:

- store one row per completed advisory request;
- preserve the user question;
- record selected agents;
- store the final recommendation;
- support future `/history` features.

Agent_Responses Sheet

Headers:

```
response_id
conversation_id
agent_type
agent_name
agent_objective
agent_response
confidence
key_assumptions
created_at
execution_id
```

The `agent_response` field stores the specialist output as a JSON string.

Recommended expression:

```
{{
  JSON.stringify({
    executive_summary:
      $json.executive_summary
      || $json.output?.executive_summary
      || '',

    key_findings:
      $json.key_findings
      || $json.output?.key_findings
      || [],

    recommendations:
      $json.recommendations
      || $json.output?.recommendations
      || [],
```

```
    key_assumptions:
      $json.key_assumptions
      || $json.output?.key_assumptions
      || [],

    critical_questions:
      $json.critical_questions
      || $json.output?.critical_questions
      || [],

    strongest_insight:
      $json.strongest_insight
      || $json.output?.strongest_insight
      || '',

    confidence:
      $json.confidence
      ?? $json.output?.confidence
      ?? 0
  })
}}
```

Activity_Log Sheet

Headers:

```
activity_id
conversation_id
chat_id
workflow_name
action
result
details
execution_id
created_at
```

Example record:

```
activity_id:
ACT-BRD-1785500000000

action:
Multi-agent business analysis completed
```

```
result:
success

details:
Boardroom analysis completed using 4 specialist agents: market, strategy,
finance, risk.
```

Project Setup

Requirements

You need:

- an n8n instance;
- Docker Desktop;
- a Telegram account;
- a Telegram bot token;
- a Gemini API key;
- Google Sheets credentials;
- a public HTTPS webhook URL;
- Git for version control.

Telegram Bot Setup

Create the bot

1. Open Telegram.
2. Search for `@BotFather`.
3. Send:

```
/newbot
```

1. Enter a display name:

```
Boardroom AI
```

1. Enter an available username ending in `bot`.
2. Copy the bot token.

Example token format:

```
1234567890:AAExampleValue
```

Do not expose the token publicly.

Add the credential to n8n

In n8n:

```
Credentials
→ Add Credential
→ Telegram API
```

Credential name:

```
Telegram — Boardroom AI
```

Paste the bot token and save.

Gemini API Setup

1. Open Google AI Studio.
2. Create or select a Google Cloud project.
3. Create a Gemini API key.
4. Copy the key.
5. Open n8n Credentials.
6. Add:

```
Google Gemini(PaLM) API
```

1. Paste the API key.
2. Save the credential.
3. Select the credential in every Gemini Chat Model node.

Recommended initial model configuration:

```
Temperature: 0 to 0.2
```

Suggested output token limits:

```
Supervisor: 700
Specialist: 800–1,200
Executive Editor: 1,200–1,500
```

Google Sheets Setup

1. Create the `Boardroom AI Database` workbook.
2. Add the required sheets.
3. Add the headers exactly as documented.
4. Create a Google Sheets OAuth2 credential in n8n.
5. Select the credential in each Google Sheets node.
6. Test one append operation before running the full workflow.

Local n8n Setup

Example `docker-compose.yml`:

```
services:
  n8n:
    image: n8nio/n8n:latest
    container_name: boardroom-n8n
    restart: unless-stopped
    ports:
      - "5678:5678"
    environment:
      - N8N_HOST=localhost
      - N8N_PORT=5678
      - N8N_PROTOCOL=http
      - GENERIC_TIMEZONE=Africa/Lagos
      - TZ=Africa/Lagos
      - N8N_RUNNERS_ENABLED=true
      - N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS=true
    volumes:
      - n8n_data:/home/node/.n8n

  task-runners:
    image: n8nio/runners:latest
    container_name: boardroom-task-runners
    restart: unless-stopped
```

```
    environment:
      - N8N_RUNNERS_TASK_BROKER_URI=http://n8n:5679
      - N8N_RUNNERS_AUTH_TOKEN=replace-with-a-secure-token
    depends_on:
      - n8n

volumes:
  n8n_data:
```

Start:

```
docker compose up -d
```

Open:

```
http://localhost:5678
```

Webhook Configuration

Telegram requires a publicly reachable HTTPS webhook.

A local URL such as:

```
http://localhost:5678
```

cannot receive Telegram webhook events.

Temporary Cloudflare Tunnel

Install Cloudflared:

```
winget install --id Cloudflare.cloudflared
```

Start a temporary tunnel:

```
cloudflared tunnel --url http://localhost:5678
```

Cloudflare returns a URL similar to:

```
https://example-random-name.trycloudflare.com
```

Add it to the n8n service:

```
environment:
  - WEBHOOK_URL=https://example-random-name.trycloudflare.com/
  - N8N_EDITOR_BASE_URL=https://example-random-name.trycloudflare.com/
  - N8N_PROXY_HOPS=1
```

Restart n8n:

```
docker compose down
docker compose up -d
```

A temporary tunnel URL changes whenever Cloudflared restarts.

For a permanent showcase deployment, use:

- a named Cloudflare Tunnel;
- a VPS;
- Railway;
- Render;
- a managed n8n instance;
- another HTTPS-capable hosting environment.

Building the Specialist Sub-Workflow

The specialist workflow expects these fields:

```
conversation_id
agent_type
agent_name
agent_objective
user_request
request_type
analysis_depth
context
```

Python input convention

This project uses n8n external Python runners.

Read the current input with:

```
data = _items[0]["json"]
```

For several items:

```
for item in _items:
    data = item["json"]
```

Return one item:

```
return [{"json": output}]
```

Return several items:

```
return [
    {"json": first_output},
    {"json": second_output}
]
```

Do not use:

```
_item
_json
_input
$("Node Name")
```

`$("Node Name")` is JavaScript-style n8n expression syntax and is invalid inside Python Code nodes.

Use Edit Fields nodes to restore data from earlier named nodes.

Building the Telegram Orchestrator

Normalise Telegram Update

Extract:

```
chat_id
user_id
username
incoming_message_id
message_text
command
supported_message
```

Prepare Conversation

Create:

```
conversation_id
request_mode
user_request
created_at
execution_id
```

Example conversation ID:

```
BRD-20260731110530-123456789
```

Validate Supervisor Plan

The Python validation node should:

- confirm structured output;
- remove invalid agents;
- remove duplicate agents;
- create a fallback when fewer than two agents are selected;
- ensure each agent has an objective;
- normalise analysis depth.

Expand Selected Agents

The expansion node converts one supervisor plan into several items.

Example input:

```
{
  "selected_agents": [
    "strategy",
    "finance"
  ]
}
```

Example output:

```
Item 1:
agent_type = strategy

Item 2:
agent_type = finance
```

Structured Output Schemas

Supervisor Schema

```
{
  "type": "object",
  "properties": {
    "request_type": {
      "type": "string"
    },
    "analysis_depth": {
      "type": "string",
      "enum": [
        "quick",
        "standard",
        "deep"
      ]
    },
    "problem_summary": {
      "type": "string"
```

```
    },
    "selected_agents": {
      "type": "array",
      "items": {
        "type": "string",
        "enum": [
          "market",
          "strategy",
          "finance",
          "risk"
        ]
      }
    },
    "agent_tasks": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "agent_type": {
            "type": "string",
            "enum": [
              "market",
              "strategy",
              "finance",
              "risk"
            ]
          },
          "objective": {
            "type": "string"
          }
        },
        "required": [
          "agent_type",
          "objective"
        ],
        "additionalProperties": false
      }
    },
    "key_context": {
      "type": "string"
    },
    "expected_output": {
      "type": "string"
    }
  },
  "required": [
    "request_type",
    "analysis_depth",
```

```
    "problem_summary",
    "selected_agents",
    "agent_tasks",
    "key_context",
    "expected_output"
  ],
  "additionalProperties": false
}
```

Specialist Schema

```
{
  "type": "object",
  "properties": {
    "executive_summary": {
      "type": "string"
    },
    "key_findings": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "recommendations": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "key_assumptions": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "critical_questions": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "strongest_insight": {
      "type": "string"
    },
```

```
    "confidence": {
      "type": "integer",
      "minimum": 0,
      "maximum": 100
    }
  },
  "required": [
    "executive_summary",
    "key_findings",
    "recommendations",
    "key_assumptions",
    "critical_questions",
    "strongest_insight",
    "confidence"
  ],
  "additionalProperties": false
}
```

Rate-Limit Handling

A single user request generates several Gemini API calls.

Quick mode

```
Supervisor Planner: 1
Specialist Agent 1: 1
Specialist Agent 2: 1
Executive Editor: 1

Total: approximately 4 requests
```

Deep mode

```
Supervisor Planner: 1
Four Specialist Agents: 4
Executive Editor: 1

Total: approximately 6 requests
```

Retries or parser repairs may increase the total.

Sequential specialist execution

To reduce request bursts:

```
08 Expand Selected Agents
        ↓
08B Loop Over Items
        ↓
08C Wait
        ↓
09 Execute Specialist Sub-workflow
        ↓
Return to Loop Over Items
```

Recommended settings:

```
Batch Size: 1
Wait: 8–15 seconds
```

Retry settings

For AI nodes:

```
Retry On Fail: Enabled
Max Tries: 3
Wait Between Tries: 10,000 ms
```

Do not configure malformed AI responses to simply continue through the workflow. Later nodes require valid fields.

Reduce token usage

Use concise prompts and output limits.

Recommended:

```
Supervisor: 700 tokens
Specialists: 1,000 tokens
Executive Editor: 1,300 tokens
```

Testing

Test 1: Greeting

Send:

```
hello
```

Expected:

- message classified as `greeting`;
- greeting response sent;
- Supervisor Planner does not run.

Test 2: Help request

Send:

```
What can you do?
```

Expected:

- message classified as `help_request`;
- capability message sent;
- no Gemini request.

Test 3: Quick analysis

Send:

```
/quick Help me create pricing packages for an AI automation consulting service
targeting small Nigerian businesses.
```

Expected:

```
Supervisor output: 1 item
Expanded agents: 2 items
Specialist results: 2 items
Agent_Responses rows: 2
Aggregated response: 1 item
Telegram final reply: 1
```

```
Conversation row: 1
Activity row: 1
```

Test 4: Deep analysis

Send:

```
/deep I want to launch a laundry pickup and delivery service in Lagos. Analyse
the market, strategy, finances, risks and launch plan.
```

Expected:

```
Selected agents:
market
strategy
finance
risk

Specialist results: 4
Agent_Responses rows: 4
Final Telegram response: 1
```

Test 5: Unsupported message

Send a blank or non-text update.

Expected:

- unsupported branch;
- guidance message;
- no AI call.

Test 6: Rate limit

Send several deep requests rapidly.

Expected possible result:

```
429 RESOURCE_EXHAUSTED
```

Then confirm:

- specialist calls are sequential;
- wait node is active;
- retries are enabled;
- output limits are reduced.

Sample Requests

Market opportunity

```
/deep I want to build a home cleaning marketplace for working professionals in Lagos. Analyse demand, competition, pricing, risks and launch strategy.
```

Pricing

```
/quick Help me price an AI automation consulting service for SMEs.
```

Product launch

```
Analyse a subscription-based financial dashboard for small retail businesses.
```

Risk analysis

```
/deep Challenge my plan to launch a motorcycle delivery company. Identify the main failure points and mitigations.
```

Expansion strategy

```
We operate a small software agency and want to expand into another African country. What should we consider?
```

Expected Output

Example final response:

🏛️ BOARDROOM VERDICT

🎯 Opportunity

The service addresses a genuine convenience problem, but the model should begin with a tightly defined service area rather than attempting city-wide coverage.

🔍 Key Findings

• Working professionals and busy households are the clearest initial customer segments.
• Reliability, garment handling and turnaround time will matter more than broad service variety.
• WhatsApp is suitable for the first version, while a dedicated platform can follow after demand is validated.

💰 Commercial View

Start with simple package pricing and clearly defined delivery zones. All cost estimates should be validated using actual local transport, labour and cleaning-partner quotations.

⚠️ Major Risks

• Inconsistent service quality
• Failed or delayed pickups
• Customer acquisition costs
• Damage or lost garments

Use standard operating procedures, partner scorecards and clear compensation policies.

🗺️ Execution Roadmap

1. Select one pilot area.
2. Interview 20 potential customers.
3. Sign two or three cleaning partners.
4. Run a four-week WhatsApp-based pilot.
5. Measure repeat usage, margins and complaints.

✅ Final Recommendation

VALIDATE FIRST.

The concept is promising, but launch only after testing customer willingness to pay and delivery economics.

```
Average specialist confidence: 82%
```

Error Handling

Model output does not fit required format

Cause:

- Gemini returned conversational text;
- required fields were missing;
- the schema was too complex;
- a greeting reached the Supervisor Planner;
- the model returned invalid enum values.

Fix:

- route greetings before the Supervisor;
- simplify the schema;
- reduce temperature;
- explicitly request all required fields;
- validate the output in Python;
- add safe fallback agents.

Invalid Python syntax

Cause:

JavaScript-style node access was used inside Python:

```
$("05B Restore Conversation Context")
```

Fix:

Use an Edit Fields node before the Python node and access:

```
data = _items[0]["json"]
```

Telegram webhook requires HTTPS

Error:

```
Bad Request: bad webhook: An HTTPS URL must be provided for webhook
```

Fix:

- use Cloudflare Tunnel;
- configure `WEBHOOK_URL`;
- restart n8n;
- reactivate the Telegram Trigger.

Telegram message not found

Cause:

The workflow used the user's incoming message ID rather than the bot's progress-message ID.

Fix:

Use the ID returned by:

```
05 Send Progress Message
```

Telegram formatting error

Cause:

Generated text contains invalid Telegram Markdown.

Fix:

```
Parse Mode: None
```

Empty agent response

Cause:

Sub-workflow output may be nested inside `output`.

Use fallbacks:

```
$json.executive_summary
|| $json.output?.executive_summary
|| ''
```

The cleanest fix is to return a flat structure from the final sub-workflow node.

Gemini too many requests

Error:

```
The service is receiving too many requests from you
```

Fix:

- use Loop Over Items;
- execute one specialist at a time;
- add an 8–15 second delay;
- enable retries;
- reduce token usage;
- check active Gemini project quotas.

Security

Never commit:

```
Telegram bot tokens
Gemini API keys
Google OAuth credentials
n8n encryption keys
runner authentication tokens
Google Sheet IDs containing private data
real Telegram user IDs
private business requests
production webhook URLs
```

Recommended `.gitignore`

```
.env
.env.*
!.env.example
```

```
credentials/
secrets/
private/
data/

*.key
*.pem
*.p12
*.crt

node_modules/
__pycache__/
*.pyc

.idea/
.vscode/

.DS_Store
Thumbs.db

n8n_data/
```

Workflow exports

Before publishing workflow JSON:

1. Remove or replace personal email addresses.
2. Remove real Telegram usernames and chat IDs.
3. Remove test conversations containing private information.
4. Confirm API keys are stored only in credentials.
5. Replace Google Sheet IDs where necessary.
6. Remove production webhook URLs.
7. Use fictional sample requests.

Repository Structure

```
boardroom-ai/
├── README.md
├── LICENSE
├── .gitignore
├── .env.example
├── docker-compose.yml
```

```
│
├── workflows/
│   ├── boardroom-telegram-orchestrator.json
│   ├── boardroom-specialist-agent.json
│   └── boardroom-error-handler.json
│
├── docs/
│   ├── architecture.md
│   ├── setup.md
│   ├── telegram-setup.md
│   ├── gemini-setup.md
│   ├── schemas.md
│   ├── testing.md
│   ├── rate-limits.md
│   └── troubleshooting.md
│
├── assets/
│   ├── boardroom-workflow.png
│   ├── specialist-workflow.png
│   ├── telegram-example.png
│   └── google-sheets-log.png
│
├── sample-data/
│   ├── telegram-update.json
│   ├── supervisor-output.json
│   ├── specialist-input.json
│   ├── specialist-output.json
│   └── executive-response.txt
│
└── templates/
    ├── google-sheets-headers.csv
    ├── supervisor-schema.json
    ├── specialist-schema.json
    └── environment.example
```

Known Limitations

- AI outputs depend on the quality of the user’s prompt.
- The system does not automatically verify market statistics.
- Financial figures are assumptions unless external data is provided.
- Google Gemini quotas may limit rapid multi-agent execution.
- Google Sheets is not suitable for high-volume transactional storage.
- Temporary Cloudflare Tunnel URLs change after restart.
- Telegram message length limits require concise final responses.

- The first version does not use long-term conversational memory.
- The first version does not perform live web research.
- Inline callback buttons may require a separate callback-query workflow.
- Specialist agents share the same language model and differ mainly by prompt and objective.
- A failed specialist request may prevent complete aggregation unless fallback handling is added.

Future Improvements

Inline callback actions

Add buttons such as:

```
Challenge This Plan
Expand Financial View
Show Risks
Create Roadmap
Start New Analysis
```

Conversation history

Add:

```
/history
```

The bot could retrieve previous conversations from Google Sheets.

Follow-up questions

Allow users to ask:

```
Expand the financial assumptions.
```

The workflow would retrieve the previous conversation and selected agent outputs.

Web research agent

Add a research specialist capable of retrieving:

- market statistics;
- competitor information;

- regulations;
- pricing benchmarks;
- public reports.

Source verification

Add:

- citation storage;
- source-quality scoring;
- claim verification;
- confidence by finding.

Document generation

Generate:

- PDF boardroom reports;
- Google Docs;
- business-plan summaries;
- investor memos;
- strategy roadmaps.

Voice input

Accept Telegram voice notes and convert them to text before analysis.

Image and document input

Allow users to upload:

- business plans;
- financial models;
- pitch decks;
- market reports;
- spreadsheets.

Dedicated memory database

Replace Google Sheets with:

- PostgreSQL;
- Supabase;
- Redis;
- a vector database.

User access control

Add:

- approved Telegram user IDs;
- daily request limits;
- paid user tiers;
- admin controls.

Usage monitoring

Track:

```
model_calls
input_tokens
output_tokens
estimated_cost
execution_duration
rate_limit_errors
```

Automatic fallback models

When Gemini quota is reached, route to another configured provider.

Agent debate

Add a second review stage where specialists critique each other before the Executive Editor runs.

Portfolio Value

Boardroom AI demonstrates more than a basic chatbot.

It showcases:

- event-driven workflow automation;
- Telegram integration;
- webhook configuration;
- multi-agent architecture;
- supervisor routing;
- dynamic task delegation;
- reusable sub-workflows;
- structured LLM responses;

- Python validation;
- one-to-many item expansion;
- many-to-one aggregation;
- executive synthesis;
- persistent logging;
- rate-limit engineering;
- operational error handling;
- modular workflow design.

Suggested portfolio description:

> Boardroom AI is a Telegram-based multi-agent advisory platform built with n8n and Google Gemini. A Supervisor Agent classifies business requests and dynamically assigns Market, Strategy, Finance and Risk specialists. Their structured responses are aggregated and synthesised by an Executive Editor into a practical recommendation, while all conversations and agent outputs are logged in Google Sheets.

License

Author

Raimi Azeez Babatunde

Data Scientist, Python Developer and AI Automation Builder.

Areas of interest:

- AI automation;
- multi-agent systems;
- financial technology;
- data science;
- workflow orchestration;
- quantitative systems;
- Python development;
- n8n automation;
- business process automation.

Disclaimer

Boardroom AI provides automated advisory analysis for educational, portfolio and decision-support purposes.

The generated recommendations do not constitute professional financial, legal, investment, regulatory, tax or business advice.

Users should independently verify market claims, regulations, costs, financial assumptions and implementation requirements before making business decisions.